DATE: PAGE:

TRIGGERS

A trigger in MySQL is a special type of stored program that automatically executes when a particular event occurs on a table. In simple words a trigger automatically performs an action when data is inserted, updated, or deleted in a table.

1. When do we use triggers?

Imagine we have an employee table, whenever an employee's salary changes, we want to record that changes automatically. Instead of manually recording it every time we can create a trigger to do the work.

Triggers are used to

- Maintain audit logs
- Enforce business rules
- Prevent Invalid data
- Automatically update information
- Reduce manual work

Scanned with OKEN Scanner

DATE: PAGE:

2 Events that activate triggers
There are three main events:

a INSERT: Adds a new row to a table
Ex: adding a new employee

b UPDATE: Modifies existing data
Ex: changing an employee's salary

c DELETE: Removes a row from a table
Ex: Deleting an employee's record.

3 Types of Triggers
Triggers are classified according to when they execute and which event activates them

Trigger type	Meaning
• BEFORE INSERT	Executes before a new row is inserted
• AFTER INSERT	Executes after a new row is inserted
• BEFORE UPDATE	Executes before an existing row's updated
• AFTER UPDATE	Executes after an existing row is updated
• BEFORE DELETE	Executes before a row is deleted
• AFTER DELETE	Executs after a row is deleted

Scanned with OKEN Scanner

DATE: PAGE:

Ex: suppose the employee table contains

emp_id	emp_name	salary
101	Arun	30000
102	Priya	40000

- If you insert Rahul, an AFTER INSERT trigger executes after his row is inserted
- If priya's salary changes from ₹40,000 to ₹45,000, an AFTER UPDATE trigger executes after the changes
- If Arun's record is deleted, a BEFORE DELETE trigger executes before the row is removed

* BEFORE means before the table operation
AFTER means after the table operation

Syntax for creating a trigger

```
DELIMITER //
CREATE TRIGGER trigger_name
BEFORE/AFTER INSERT/UPDATE/DELETE
  ON table_name
  FOR EACH ROW
  BEGIN
      --- SQL statements
```

Scanned with OKEN Scanner

DATE: PAGE:

END //
DELIMITER ;

DELIMITER // : Temporarily changes the statements ending delimiter to //
CREATE TRIGGER : Creates a new trigger
trigger_name : The name you give the trigger
BEFORE / AFTER : Specifies when it executes
INSERT / UPDATE / DELETE : Specifies the event
ON table_name : specifies the table associated with the trigger
FOR EACH ROW : Executes the trigger for each affected row
BEGIN .... END : contains the SQL statements executed by the trigger
DELIMITER : Restores the normal delimiter

- NOTE : choose one timing and one event in the actual trigger definition, such as BEFORE INSERT you cannot write all the alternatives together

Example, Prevent a NULL salary
Suppose your employee salary should never contain a missing salary.

Scanned with OKEN Scanner

DATE: PAGE:

we can create BEFORE INSERT trigger that checks the salary before a new employee is added

```
DELIMITER //
  CREATE TRIGGER check_Salary
  BEFORE INSERT ON employee
   FOR EACH ROW
   BEGIN
      IF NEW.Salary is NULL THEN
         SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = "Salary cannot
                          be NULL';
   END IF;
   END//
   DELIMITER;
```

SIGNAL : raises an error
SQLSTATE '45000' : Specifies a general user defined exception

4 How to display triggers
Uses the SHOW TRIGGERS command to list triggers in the currently selected data base

Scanned with OKEN Scanner

DATE: PAGE:

SHOW TRIGGERS;

5 HOW to delete a triggers?

Use DROP TRIGGERS when no longer need a trigger

DROP TRIGGER check_salary;

Advantages

- Executes actions automatically
- Maintains audit logs
- Enforces business rules
- Reduces Manual work
- Improves data integrity

Disadvantage

- Can be difficult to debug
- Too many triggers may reduce performance
- Hidden execution can confuse developers

Scanned with OKEN Scanner